

BerryBot

pico
bricks | IDE



MicroPython



Application

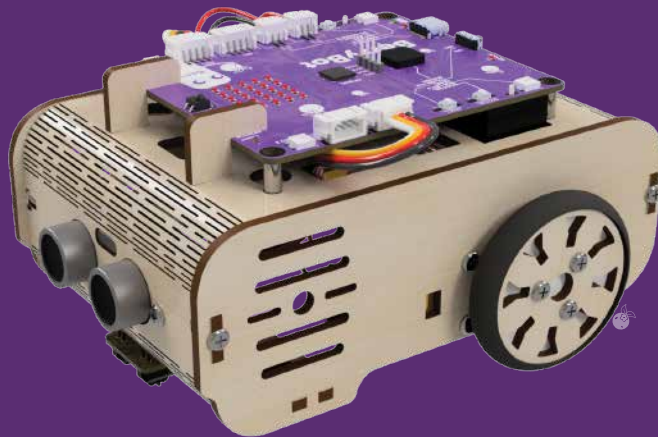


coding

light tracker mode



line tracker mode



CONTENTS

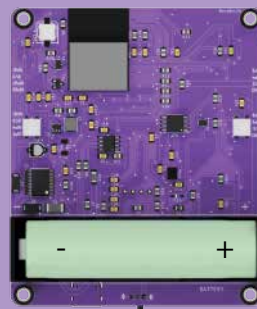
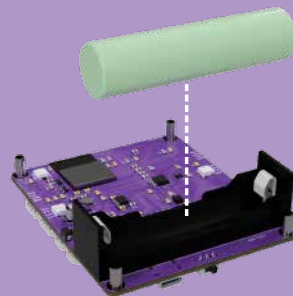
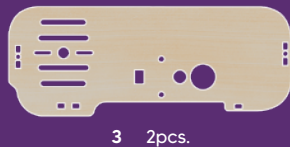
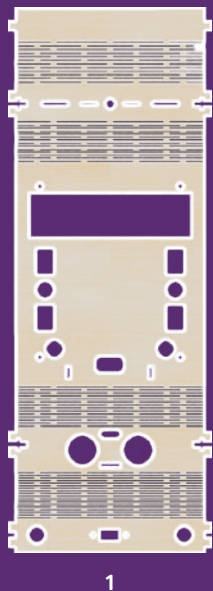
Wooden and Plexiglass Piece	03
Components of BerryBot	04
What is BerryBot?	05
Programming Environments	06
Components - Back of the Motherboard	07
Programming Platforms -What is Block Programming?	08
What is Text-Based Programming? - How to Use BerryBot?	09
Controlling BerryBot with PicoBricks Go!	10 - 12
PicoBricks Go! Menu Buttons	13
Creating a Smiley Face Emoji on Matrix LEDs	14
Block Programming	15
Lest's Start Coding With PicoBricks IDE - Blink Activity	18
Text-Based Programming	19
How to Use Thonny IDE	20 - 22
Loading the BerryBot Library	23

My First Project with BerryBot	24
Text-Based Programming	25 - 28
BerryBot Modes - Obstacle Avoidance	29 - 30
Line Tracker	34 - 38
Sumo Robot	39 - 43
Light Tracker	44 - 48
IR Remote Control	49-52
BerryBot Pinout	53

BerryBot



Wooden and Plexiglass Piece



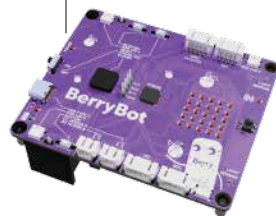
Pay attention to the battery orientation.

Components of BerryBot

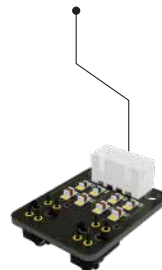
HC-SR04
Distance Sensor



BerryBot
Main Board



Line Tracer Sensor



DC Motor 2pcs.



DC Motor
Connection Bracket 2pcs.



Remote Controller



Mini Drunk Wheel
2pcs.



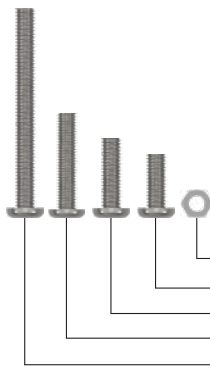
Wheel Gasket 2pcs.



Connecting Cables 4pcs.



- M3 Nut 16pcs.
- M3 x 8mm Screw 8pcs.
- M3 x 10mm Screw 4pcs.
- M3 x 15mm Screw 8pcs.
- M3 x 30mm Screw 4pcs.



What is BerryBot?

BerryBot is a robotics programming kit aimed at developing children's skills in robotics infrastructure, basic programming, hand-eye coordination, problem-solving and more. Thanks to the main board included in the kit, motor connections that allow the robot to move physically can be made quickly. With its easy-to-assemble wooden body, the mechanical and electronic setup of the robot can be completed in a short time.

Once you have completed the setup of BerryBot, you can improve your programming skills by programming it using the PicoBricks IDE either in a block-based or text-based manner.

You can also control the robot from your phone or tablet via PicoBricks GO mobile application, making it a fun STEM toy.

With the tracks included in the kit, you can quickly carry out popular projects such as line following, sumo, light following, and obstacle-avoiding robots using the mobile application. You can also learn how to develop the algorithms for these projects and write the code using the PicoBricks IDE. Additionally, with the other track included in the kit, you can carry out the tasks in the booklet or the tasks you create based on your imagination, organizing fun activities with your friends.



Programming Environments

BerryBot can be programmed online with the PicoBricks IDE using block-based and text-based coding. It can also be programmed offline using text-based coding with Thonny IDE and Arduino IDE.



RGB LEDs



Horn



Obstacle Avoidance Mode



Line Tracker Mode



Light Tracker Mode

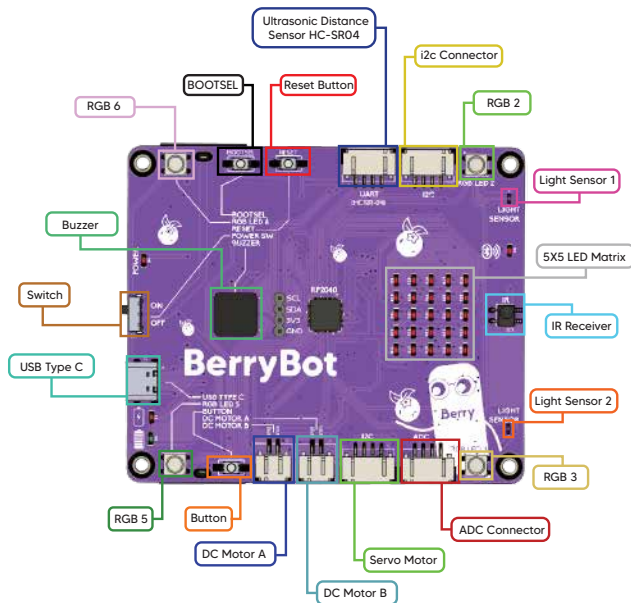


Sumo Robot

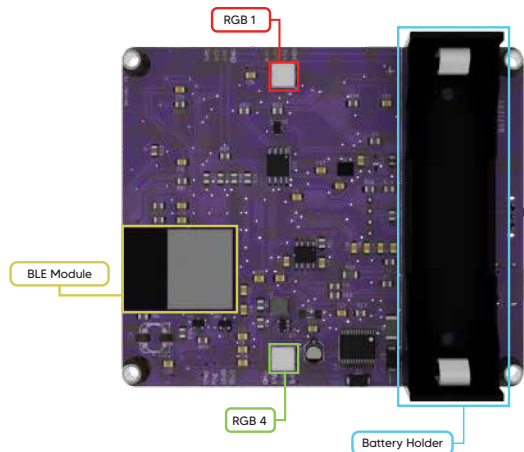


Components

Front of the Motherboard



Back of the Motherboard



BerryBot features over 25 input and output sensors on its motherboard. With its components, BerryBot becomes a project development robot that you can customize by developing your own code.

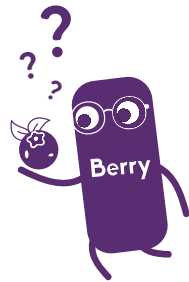
Programming Platforms

BerryBot supports both block-based and text-based programming editors. BerryBot can be programmed online with block-based and text-based coding using the PicoBricks IDE. Additionally, you can develop projects in MicroPython and C programming languages with text-based coding using editors like Thonny IDE and Arduino IDE, which you can download to your computer.

What is Block Programming?

Block-based programming is a type of visual programming designed for beginners. In block-based programming, the desired algorithm is created by dragging and dropping blocks in the editor, either stacking them vertically or nesting them. Block-based programming eliminates syntax (punctuation) errors commonly encountered in programming languages and allows for the quick and error-free creation of the desired algorithm's code.

BerryBot can be programmed using block-based programming with the PicoBricks IDE.



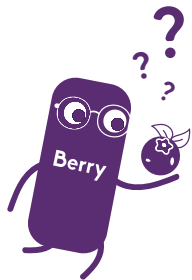
What is Text-Based Programming?

Text-based programming involves creating programs using command structures written in words. Each programming language has its own syntax rules. **C**, **C++**, and **Python** are examples of **text-based** programming languages.

BerryBot can be programmed using **text-based** programming languages with **PicoBricks IDE - PicoPy (MicroPython)**, **Arduino IDE (C)**, and **Thonny IDE (MicroPython)**.

How to Use BerryBot?

BerryBot can be controlled remotely with PicoBricks Go! with the modes available in PicoBricks Go!, you can run pre-made projects or control BerryBot using a joystick.



Controlling BerryBot with PicoBricks Go!



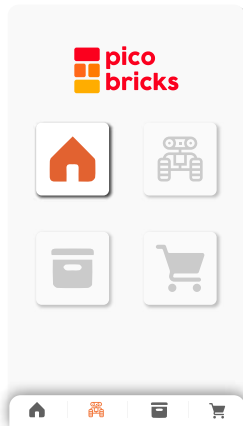
PicoBricks GO! is a mobile application that allows you to control BerryBot The REX 8 in 1 Robot Kit and PicoBricks.



Download and open PicoBricks Go! from your app store according to your phone's operating system.

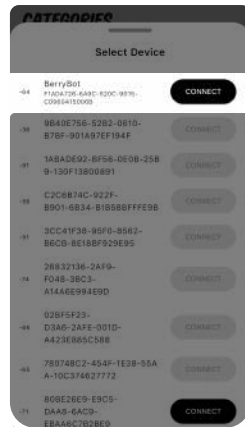


1. When you first open the app, click on the home icon in the main menu that appears.



2. Select BerryBot from the categories.

3. In the "Select Device" menu, choose BerryBot. This process will enable your BerryBot to communicate with the app via Bluetooth.



4. In the "Select Service" menu, select the second default option. This will allow you to control BerryBot remotely via Bluetooth.



5. After completing these steps, you can use the gamepad screen that opens to control BerryBot or switch to the pre-made modes.

PicoBricks Go! Menu Buttons



Allows you to return to the previous screen.



Switches to the BerryBot gamepad screen.



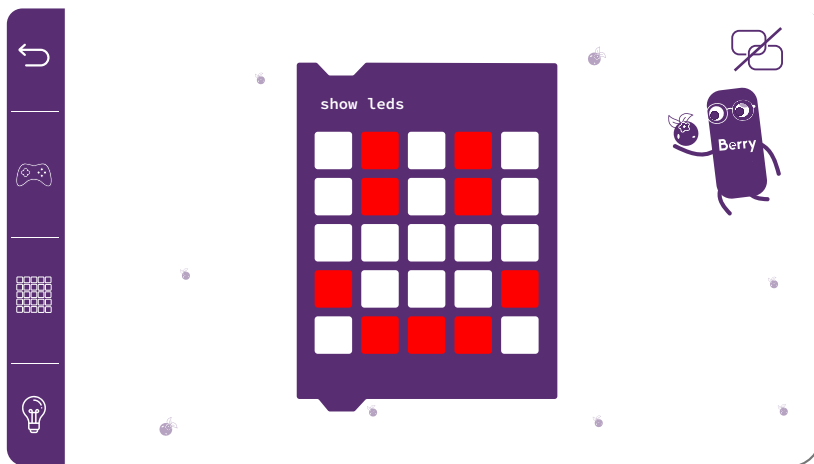
Switches to the screen where you can control BerryBot's matrix LEDs.



Switches to the screen where you can control BerryBot's RGB Leds.

Creating a Smiley Face Emoji on Matrix LEDs

Switch to the matrix LED screen using the menu buttons and create a smiley face by tapping on the LEDs screen as shown below.



Block Programming

Programming with PicoBricks IDE

PicoBricks IDE is an online programming editor that you can use from your browser on a computer with an internet connection, without installing any software on your computer. Using the editors within PicoBricks, you can create block-based programs or text-based programs in the MicroPython language.

PicoBlockly

pico
bricks IDE

Let's open <https://picobricks.com/> in the browser.



picobricks.com

A

B

Click "IDE" button.

rbt.ist/ide



Educational + Be Our Ambassador

IDE



PicoJR

A simple and visual-based coding platform for kids who are new to robotic coding

Go



PicoBlockly

Easily code PicoBricks with block coding and see python code at the same time.

Go



PicoPY

Do high-level coding with text-based python.

Go



Simulator

With PicoBricks Simulator you can experience using PicoBricks.

Go

Click "PicoBlockly" button.

Block



PicoBricks



PicoBricks Simulator

Block



PicoBricks



PicoBricks Simulator

Upload Firmware

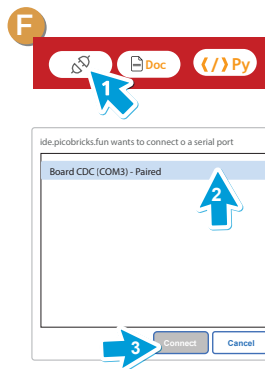
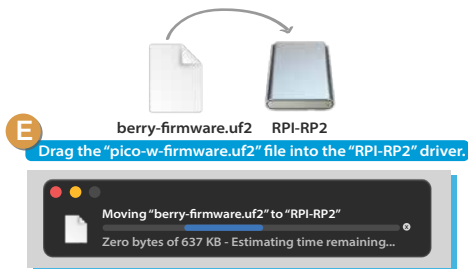
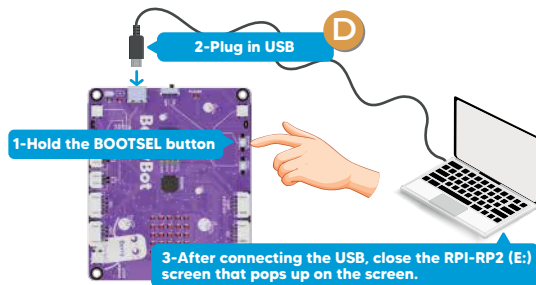
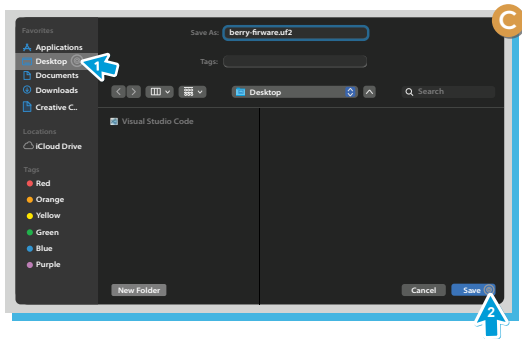
Upload Library

Upload BerryBot's Firmware

Upload BerryBot's Library

Upload Code For Mobile

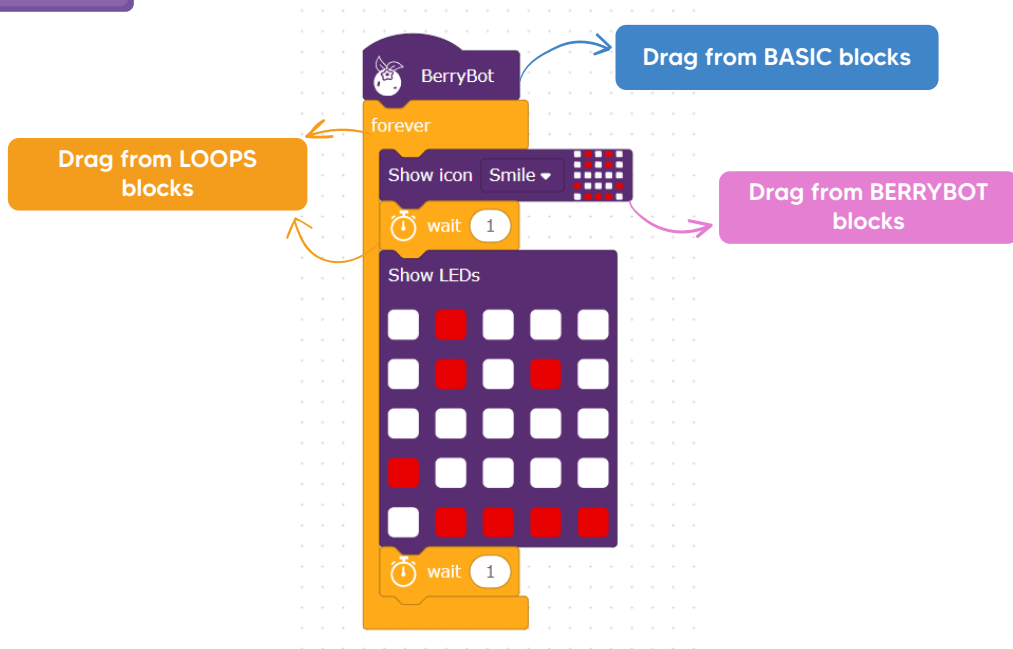
Click "Upload BerryBot's Firmware" button.



After making the connection, the connection icon will change.

Lest's Start Coding With PicoBricks IDE

Blink Activity



Text-Based Programming

Programming with Thonny IDE

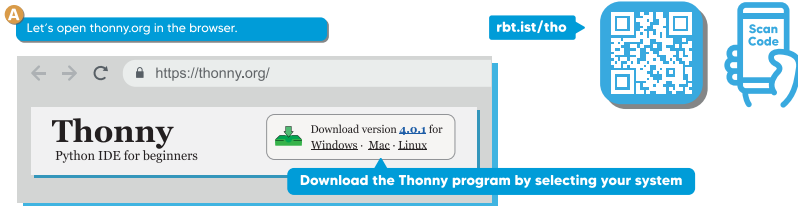
Thonny IDE is a text-based programming platform designed for the Python programming language. Using Thonny IDE, you can upload code to microcontrollers using the MicroPython programming language.

What is MicroPython?

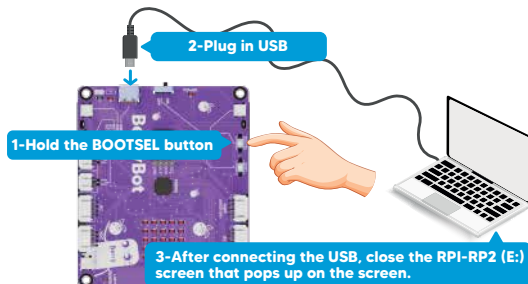
MicroPython is a software implementation designed for developing embedded software in the Python programming language. With MicroPython, you can upload code to microcontrollers using the Python programming language.

The syntax structure of MicroPython is the same as the Python programming language. Using editors like Thonny IDE, the prepared lines of code can be easily uploaded to the microcontrollers supported by the editors.

How to Use Thonny IDE

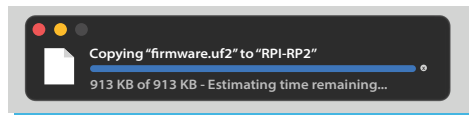
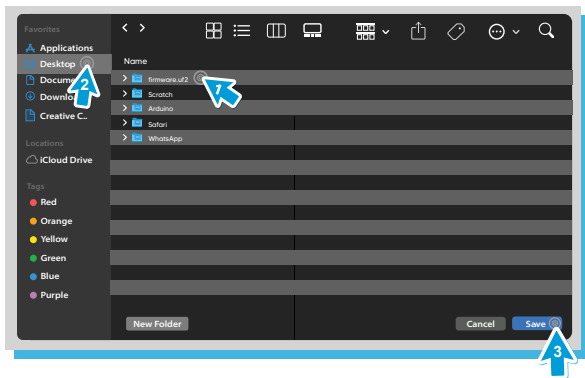


1. To put your device into boot mode, press the "BOOTSEL" button on the board and connect the cable.



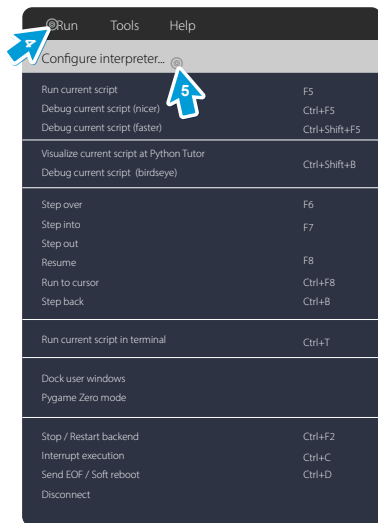
2. Go to the BerryBot GitHub page

<https://github.com/Robotistan/BerryBot/tree/main/Bootloader>
and download the file named "**firmware.uf2**".



3. Follow the steps below to upload the downloaded file to your processor.

4. Open Thonny IDE and click on the run option from the top menu.

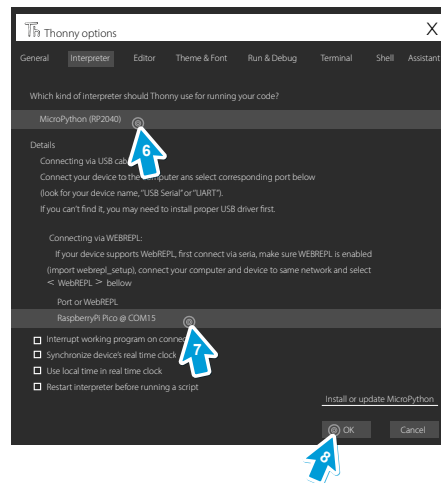


5. From the window that opens, select the **"Configure interpreter..."** option.

6. Firstly select MicroPython (RP240) from the window that opens

7. Then select the port to which your board is connected.

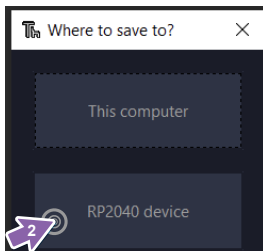
8. Finally, click the **"OK"** button.



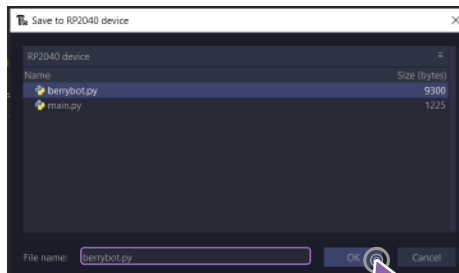
9. After the connection is complete, write the following lines of code to run your first project.

Loading the BerryBot Library

Before writing the project code, load the **"berrybot.py"** library onto the BerryBot motherboard. If you do not do this, you will get an error in your code.



Press the Ctrl+Shift+S key combination and then select RP2040 device option.



Save it as 'berrybot.py'.

My First Project with BerryBot

By writing the following lines of code in Thonny IDE, make the BerryBot RGB LEDs light up in a randomly selected color one by one.

Thonny

File Edit View Run Tools Help



```
1 from time import sleep
2 from machine import Pin
3 from berrybot import WS2812
4 import time
5 import random
6
7 ws2812 = WS2812(num_leds = 6, pin_num = 6, brightness = 0.2)
8
9 while True:
10     r=(random.randint(1, 255))
11     g=(random.randint(1, 255))
12     b=(random.randint(1, 255))
13     for i in range (6):
14         ws2812.pixels_set(i,(r,g,b))
15         ws2812.pixels_show()
16         time.sleep(1)
17
```

Text-Based Programming

What is Arduino?

The Integrated Development Environment (IDE) for Arduino is a cross-platform application written in C and C++ languages (for Linux, macOS, Windows). It is used to write and upload programs to Arduino compatible boards, but can also be used in 3rd party cores and vendor development boards.

How to Use Arduino IDE?

A

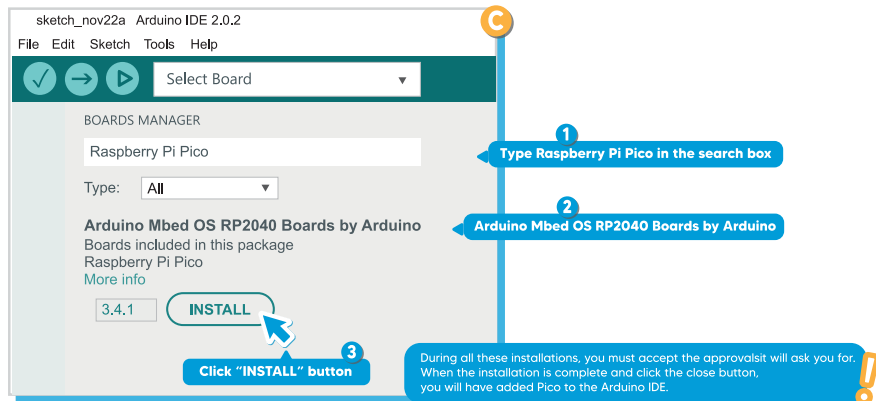
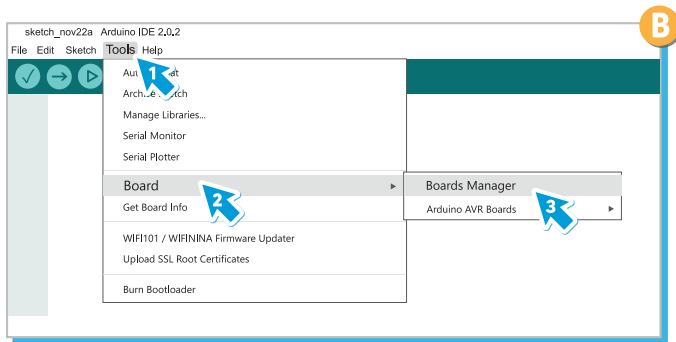
Let's open <https://www.arduino.cc/en/software> in the browser.

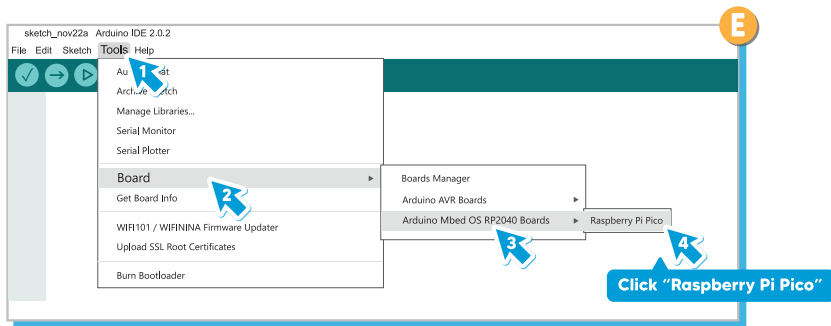
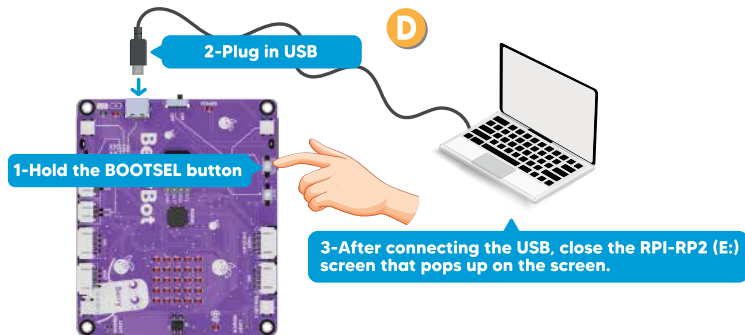


rbt.isst/ardu



Download the Arduino IDE program
by selecting your system





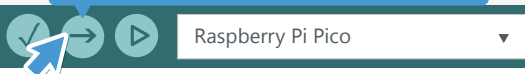
You are ready now.
Let's start coding
with Arduino

Do your first coding with Arduino IDE

Activating the Buzzer with the BerryBot Horn Activity

Arduino IDE

File **Click the "Upload" button after typing the code**



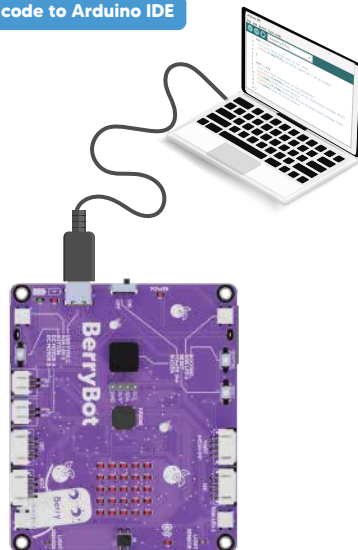
```
1 #include <Wire.h>
2
3 #define MODE_BUTTON 10
4 #define BUZZER_PIN 14
5
6 int buttonState=0;
7
8 void setup() {
9   Serial.begin(115200);
10  Wire.begin();
11  pinMode(BUZZER_PIN, OUTPUT);
12  pinMode(MODE_BUTTON, OUTPUT);
13 }
14
15
16 void loop() {
17   buttonState = digitalRead(MODE_BUTTON);
18   delay(100);
19   if (buttonState==1){
20     tone(BUZZER_PIN, 500);
21   }
22   else{
23     noTone(BUZZER_PIN);
24   }
25 }
26
```

Texts starting with the // sign are comments.
You do not need to write.

[rbt.isr/blink](https://rbt.isr.io/rbt.isr.io/blink)



1 Type the code to Arduino IDE

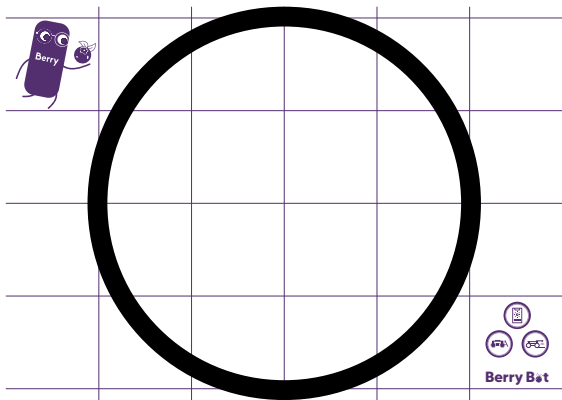


BerryBot Modes

BerryBot comes with 4 pre-made modes: obstacle avoidance, sumo robot, line following, and light following. You can quickly run these modes and perform tasks on the track using the buttons in the PicoBricks Go! application or by using the ready-made PicoBricks IDE and Thonny IDE (MicroPython) codes.

Obstacle Avoidance

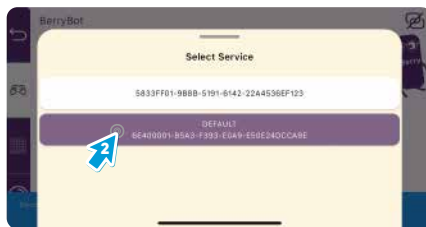
In obstacle avoidance mode, BerryBot detects objects within 12 cm in front of it using its front ultrasonic distance sensor and avoids them.



You can create a maze for BerryBot to navigate by using the checkered areas on the track.



To run the obstacle avoidance mode in the PicoBricks Go! mobile application, you can follow the steps below.



The code blocks for the obstacle avoidance mode prepared using PicoBricks IDE are given in the image below.



After the button is pressed, continuously repeat the following steps.

If there is no object within 12 cm in front of BerryBot, move forward at 100% power.

Otherwise, if there is an object within 12 cm in front of BerryBot, perform the following movement combination:

- Stop for 0.5 seconds
- Move backward for 0.3 seconds
- Stop for 0.2 seconds
- Turn left for 0.6 or 1 seconds
- Stop for 0.2 seconds



berrybot_sonic_mode.py >

```
1 #####Libraries#####
2 from machine import Pin,
3 from time import sleep
4 from berrybot import HCSR04, TB6612
5 import time
6 #####Pin Defination#####
7 MOTOR_A1_PIN = 25
8 MOTOR_A2_PIN = 24
9 MOTOR_B1_PIN = 22
10 MOTOR_B2_PIN = 23
11 MOTOR_PWM_A_PIN = 15
12 MOTOR_PWM_B_PIN = 21
13 MODE_BUTTON = 10
14 #####Pin Initialization#####
15 motor = TB6612(MOTOR_A1_PIN, MOTOR_A2_PIN, MOTOR_B1_PIN, MOTOR_B2_PIN, MOTOR_PWM_A_PIN, MOTOR_PWM_B_PIN)
16 sensor = HCSR04(trigger_pin=8, echo_pin=9, echo_timeout_us=10000)
17 push_button = Pin(MODE_BUTTON,Pin.IN,Pin.PULL_DOWN)
18 counter=0
19 right_counter=0
20 left_counter=0
21
22 while (push_button.value()==0):
23     motor.stop()
24
25 while True:
26     print(sensor.distance_cm())
27
28     if (sensor.distance_cm()) > (12):
29         motor.forward(100 * 650)
30     else:
31         motor.stop()
32         time.sleep((0.5))
33         motor.backward(100 * 650)
34         time.sleep((0.1))
35         motor.stop()
36         time.sleep((0.2))
```

rbt.is/obstacleavoidancemp



Scan the QR code to
access the code.



sonic_mode_arduino.ino

```
1  #include <Wire.h>
2  #include <stdio.h>
3
4  // Pin Defination
5  #define TX_PIN 0
6  #define RX_PIN 1
7  #define NEOPIXEL_PIN 6
8  #define TRIG_PIN 8
9  #define ECHO_PIN 9
10 #define MODE_BUTTON 10
11 #define BUZZER_PIN 14
12 #define PWM_A 15
13 #define IR_PIN 20
14 #define PWM_B 21
15 #define INPUT_B1 22
16 #define INPUT_B2 23
17 #define INPUT_A2 24
18 #define INPUT_A1 25
19 #define LEFT_SENSOR 26
20 #define RIGHT_SENSOR 27
21 #define LDR_R_PIN 28
22 #define LDR_L_PIN 29
23
24 // Variable
25 long duration;
26 int distance;
27 int left_counter=0;
28
29 void attachMotor()
30 {
31     pinMode(INPUT_A1, OUTPUT);
32     pinMode(INPUT_A2, OUTPUT);
33     pinMode(INPUT_B1, OUTPUT);
34     pinMode(INPUT_B2, OUTPUT);
35     pinMode(PWM_A, OUTPUT);
36     pinMode(PWM_B, OUTPUT);
37 }
38
```

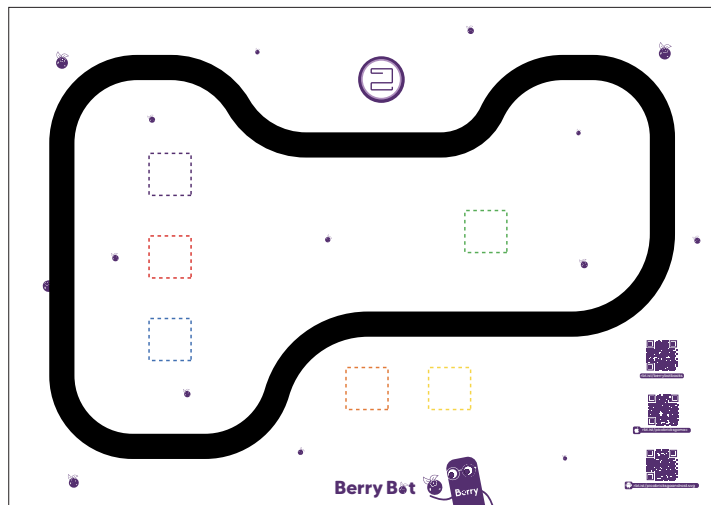
rbt.ist/obstacleavoidancearduino



Scan the QR code to
access the code.

Line Tracker

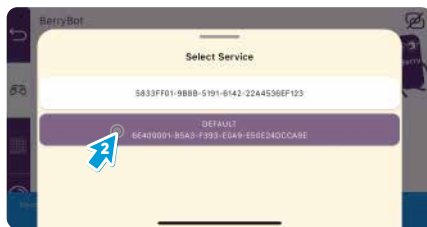
In line tracker mode, BerryBot can autonomously follow the black lines on the track using the line tracker sensor located underneath it.



You can place BerryBot on the black path on the track to make it follow the road.



To run the line following mode in the PicoBricks Go! mobile application, you can follow the steps below.

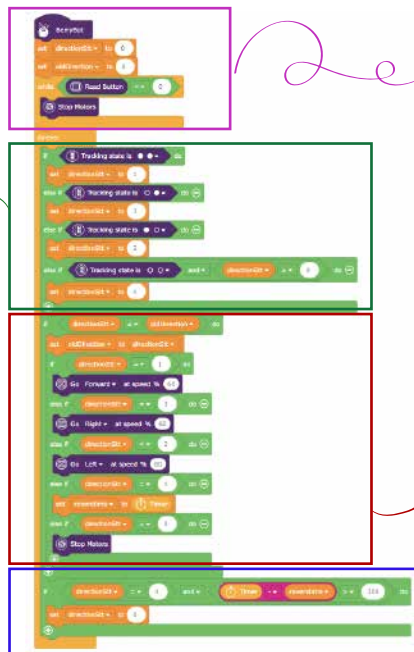


The code blocks for the line tracker mode prepared using PicoBricks IDE are given in the image below.

Let's create the condition structure necessary to determine the direction assignments based on the values detected by the line following sensor located under BerryBot. While making these assignments, we will change the value of the "directionStt" variable that we created at the beginning of our code. The numerical equivalents of the direction values are as follows:

- Stop = 0
- Forward = 1
- Left = 2
- Right = 3
- Backward = 4

Let's create the condition structure that determines the required duration value for BerryBot to move backward.



After starting our code with the BerryBot block, let's create two variables named "directionStt" and "oldDirection" to assign directions. Then, let's create a loop that keeps the BerryBot stationary until a button is pressed.

Let's create the condition structure where the movements that the motors need to perform based on the determined direction values are called.



lineTracker.py ×

```
1 #####Libraries#####
2 from time import sleep
3 from machine import Pin, PWM, UART, Timer, ADC
4 import time, utime
5 from berrybot import TB6612
6 #####Pin Defination#####
7 MOTOR_A1_PIN = 25
8 MOTOR_A2_PIN = 24
9 MOTOR_B1_PIN = 22
10 MOTOR_B2_PIN = 23
11 MOTOR_PWM_A_PIN = 15
12 MOTOR_PWM_B_PIN = 21
13 LEFT_TRACKER = 26
14 RIGHT_TRACKER = 27
15 MODE_BUTTON = 10
16
17 #####Variables#####
18 Mid_Speed = 58000
19 Low_Speed = 43000
20
21 STOP = 0
22 FWD = 1
23 LEFT = 2
24 RIGHT = 3
25 BWD = 4
26
27 threshold = 60000
28 directionStt = STOP
29 oldDirection = STOP
30
31 #####Pin Initialization#####
32 motor = TB6612(MOTOR_A1_PIN, MOTOR_A2_PIN, MOTOR_B1_PIN, MOTOR_B2_PIN, MOTOR_PWM_A_PIN, MOTOR_PWM_B_PIN)
33 leftSensor = ADC(Pin(LEFT_TRACKER))
34 rightSensor = ADC(Pin(RIGHT_TRACKER))
35 push_button = Pin(MODE_BUTTON, Pin.IN, Pin.PULL_DOWN)
36
```

rbt.ist/linetrackermmp



Scan the QR code to
access the code.



lineTracker.ino

```
1 // Libraries
2 #include <Wire.h>
3 #include <stdio.h>
4
5 // Pin Definition
6 #define MOTOR_B1 22
7 #define MOTOR_B2 23
8 #define MOTOR_A2 24
9 #define MOTOR_A1 25
10 #define PWM_A 15
11 #define PWM_B 21
12 #define LEFT_SENSOR 26
13 #define RIGHT_SENSOR 27
14 #define MODE_BUTTON 10
15
16 #define TRACKER_THRESHOLD 900
17
18 #define STOP 0
19 #define FWD 1
20 #define BWD 2
21 #define RIGHT 3
22 #define LEFT 4
23
24 // Variable
25 int leftSensor = 0;
26 int rightSensor = 0;
27 uint8_t directionStt = STOP;
28 uint8_t oldDirection = STOP;
29 unsigned long reverseTime = 0;
30
31 void attachMotor()
32 {
33   pinMode(MOTOR_A1, OUTPUT);
34   pinMode(MOTOR_A2, OUTPUT);
35   pinMode(MOTOR_B1, OUTPUT);
36   pinMode(MOTOR_B2, OUTPUT);
37   pinMode(PWM_A, OUTPUT);
38   pinMode(PWM_B, OUTPUT);
39 }
40
```

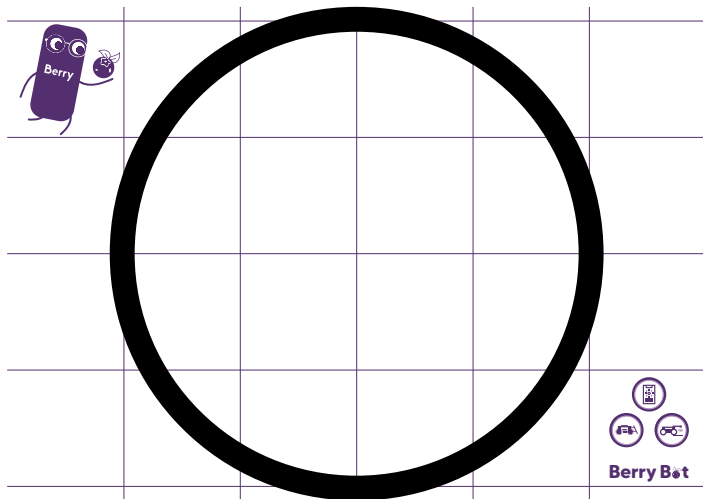
rbt.is/linetrackerarduino



Scan the QR code to
access the code.

Sumo Robot

In sumo mode, when BerryBot is placed in an area surrounded by black lines on the track, it pushes objects within the area surrounded by the black line out of the area while staying within the area itself. BerryBot uses its front distance sensor and the line following sensor underneath it for this process. Using the line following sensor, it detects the color of the ground and stays within the black-lined area of the sumo track. Using its front distance sensor, it detects objects on the track.

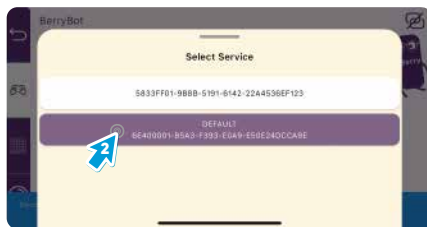


You can use the circle on the track surrounded by black lines by placing the materials that BerryBot will release and BerryBot itself inside it.

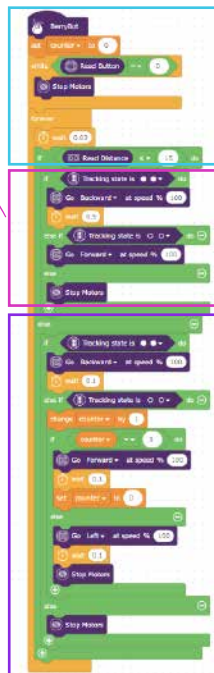


PicoBricks Go!

To run the line following mode in the PicoBricks Go! mobile application, you can follow the steps below.



The code blocks for the line tracker mode prepared using PicoBricks IDE are given in the image below.



If both receivers of BerryBot's line following sensor detect the line, let's create code blocks that make BerryBot move backward to stay within the track. If both receivers detect the white color and the distance value is less than 15 (meaning there is an object in front of it), let's create code blocks that make it move forward.

Let's start our code with the BerryBot block, then create a loop that stops BerryBot until the button is pressed. After that, let's create an infinite loop and set up the necessary conditional structure for the robot to detect objects in front of it.

If there is no object in front of BerryBot and the receivers detect the line, let's create code blocks that make it move backward. If the receivers detect the white color, let's create code blocks that make it move left. If none of these conditions are met, let's create code blocks that stop BerryBot.



sumoRobot.py x

```
1 #####Libraries#####
2 from machine import Pin, PWM, UART, Timer, ADC
3 from time import sleep
4 from berrybot import TB6612, HCSR04
5 #####Pin Defination#####
6 MOTOR_A1_PIN = 25
7 MOTOR_A2_PIN = 24
8 MOTOR_B1_PIN = 22
9 MOTOR_B2_PIN = 23
10 MOTOR_PWM_A_PIN = 15
11 MOTOR_PWM_B_PIN = 21
12 LEFT_TRACKER = 26
13 RIGHT_TRACKER = 27
14 MODE_BUTTON = 10
15 #####Variables#####
16 MotorSpeed = 50000
17 threshold = 60000
18 counter = 0
19 #####Pin Initialization#####
20 motor = TB6612(MOTOR_A1_PIN, MOTOR_A2_PIN, MOTOR_B1_PIN, MOTOR_B2_PIN, MOTOR_PWM_A_PIN, MOTOR_PWM_B_PIN)
21 sensor = HCSR04(trigger_pin=8, echo_pin=9, echo_timeout_us=10000)
22 leftSensor = ADC(Pin(LEFT_TRACKER))
23 rightSensor = ADC(Pin(RIGHT_TRACKER))
24 push_button = Pin(MODE_BUTTON, Pin.IN, Pin.PULL_DOWN)
25
26 while (push_button.value()==0):
27     motor.stop()
28
29 while True:
30     distance = sensor.distance_cm()
31     #print(distance)
32
```

rbt.list/sumorobotmp



Scan the QR code to
access the code.



sumoRobot.ino

```
1 // Libraries
2 #include <Wire.h>
3 #include <stdio.h>
4
5 // Pin Definition
6 #define MOTOR_B1 22
7 #define MOTOR_B2 23
8 #define MOTOR_A2 24
9 #define MOTOR_A1 25
10 #define PWM_A 15
11 #define PWM_B 21
12 #define LEFT_SENSOR 26
13 #define RIGHT_SENSOR 27
14 #define TRIG_PIN 8
15 #define ECHO_PIN 9
16 #define MODE_BUTTON 18
17
18 #define TRACKER_THRESHOLD 1000
19 #define HighSpeed 255
20
21 // Variable
22 long duration;
23 int distance;
24 int leftSensor = 0;
25 int rightSensor = 0;
26 int counter = 0;
27 unsigned long reverseTime = 0;
28
29 void attachMotor()
30 {
31   pinMode(MOTOR_A1, OUTPUT);
32   pinMode(MOTOR_A2, OUTPUT);
33   pinMode(MOTOR_B1, OUTPUT);
34   pinMode(MOTOR_B2, OUTPUT);
35   pinMode(PWM_A, OUTPUT);
36   pinMode(PWM_B, OUTPUT);
37 }
38
39 void Forward(int speed)
40 {
41   digitalWrite(MOTOR_A1, HIGH);
42   digitalWrite(MOTOR_A2, LOW);
43   digitalWrite(MOTOR_B1, HIGH);
44   digitalWrite(MOTOR_B2, LOW);
45   analogWrite(PWM_A, speed);
46   analogWrite(PWM_B, speed);
47 }
```

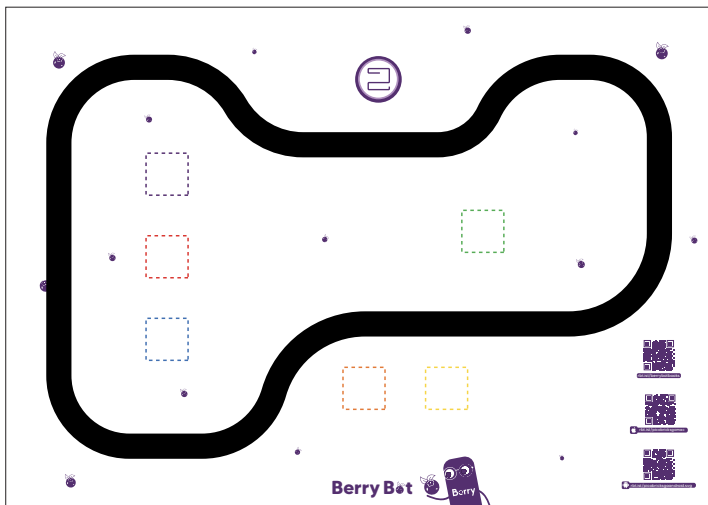
rbt.ist/sumorobotarduino



Scan the QR code to
access the code.

Light Tracker

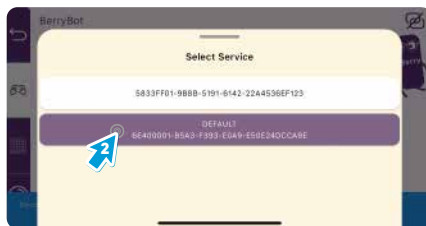
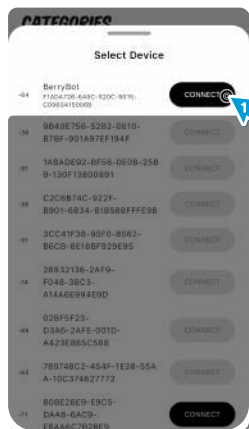
In light tracker mode, BerryBot uses the data from two light sensors on its main board to move towards the area with more light. When using this mode, you can direct any light source towards BerryBot, and it will move towards the light source.



Using the checkered areas on the track and a light source, you can move BerryBot to the desired coordinates.

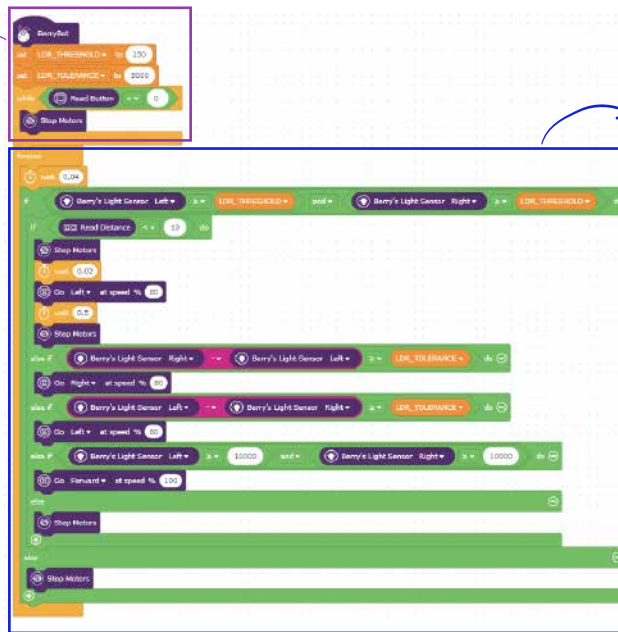


To run the line following mode in the PicoBricks Go! mobile application, you can follow the steps below.



The code blocks for the light following mode prepared using PicoBricks IDE are given in the image below.

After starting our code with the BerryBot block, let's create a loop that keeps BerryBot stopped until the button is pressed. Then, let's define two variables for the tolerance and threshold values that the light sensor will detect and assign their values.



In the marked area, the necessary condition structure for BerryBot to follow the light is created. For BerryBot to follow the light:

If the data from the left and right light sensors is greater than or equal to the threshold value:

If there is any object in front of BerryBot, perform the stop, turn left, and stop movements.

If there is no object in front of it and the data from the right light sensor is greater, move right.

If there is no object in front of it and the data from the left light sensor is greater, move left.

If none of these conditions are met, stop.



lightTracker.py x

```
1 #####Libraries#####
2 from time import sleep
3 from machine import Pin, PWM, UART, Timer, ADC
4 import time, utime
5 from berrybot import TB6612, HCSR04
6 #####Pin Defination#####
7 MOTOR_A1_PIN = 25
8 MOTOR_A2_PIN = 24
9 MOTOR_B1_PIN = 22
10 MOTOR_B2_PIN = 23
11 MOTOR_PWM_A_PIN = 15
12 MOTOR_PWM_B_PIN = 21
13 LDR_L_PIN = 29
14 LDR_R_PIN = 28
15 MODE_BUTTON = 10
16 #####Variables#####
17 LDR_THRESHOLD = 250
18 LDR_TOLERANCE = 5000
19
20 #####Pin Initialization#####
21 motor = TB6612(MOTOR_A1_PIN, MOTOR_A2_PIN, MOTOR_B1_PIN, MOTOR_B2_PIN, MOTOR_PWM_A_PIN, MOTOR_PWM_B_PIN)
22 sensor = HCSR04(trigger_pin=8, echo_pin=9, echo_timeout_us=10000)
23 ldr_left = ADC(LDR_L_PIN)
24 ldr_right = ADC(LDR_R_PIN)
25 push_button = Pin(MODE_BUTTON, Pin.IN, Pin.PULL_DOWN)
26 while (push_button.value()==0):
27     motor.stop()
28
29 while True:
30
31     LDR_L = ldr_left.read_u16()
32     LDR_R = ldr_right.read_u16()
```

rbt.ist/lighttrackemp



Scan the QR code to
access the code.



lineTracker.ino

```
1 // Libraries
2 #include <Wire.h>
3 #include <stdio.h>
4
5 // Pin Definition
6 #define MOTOR_B1 22
7 #define MOTOR_B2 23
8 #define MOTOR_A2 24
9 #define MOTOR_A1 25
10 #define PMM_A 15
11 #define PMM_B 21
12 #define LEFT_SENSOR 26
13 #define RIGHT_SENSOR 27
14 #define MODE_BUTTON 10
15
16 #define TRACKER_THRESHOLD 900
17
18 #define STOP 0
19 #define FWD 1
20 #define BWD 2
21 #define RIGHT 3
22 #define LEFT 4
23
24 // Variable
25 int leftSensor = 0;
26 int rightSensor = 0;
27 uint8_t directionStt = STOP;
28 uint8_t oldDirection = STOP;
29 unsigned long reverseTime = 0;
30
31 void attachMotor()
32 {
33     pinMode(MOTOR_A1, OUTPUT);
34     pinMode(MOTOR_A2, OUTPUT);
35     pinMode(MOTOR_B1, OUTPUT);
36     pinMode(MOTOR_B2, OUTPUT);
37     pinMode(PMM_A, OUTPUT);
38     pinMode(PMM_B, OUTPUT);
39 }
40
```

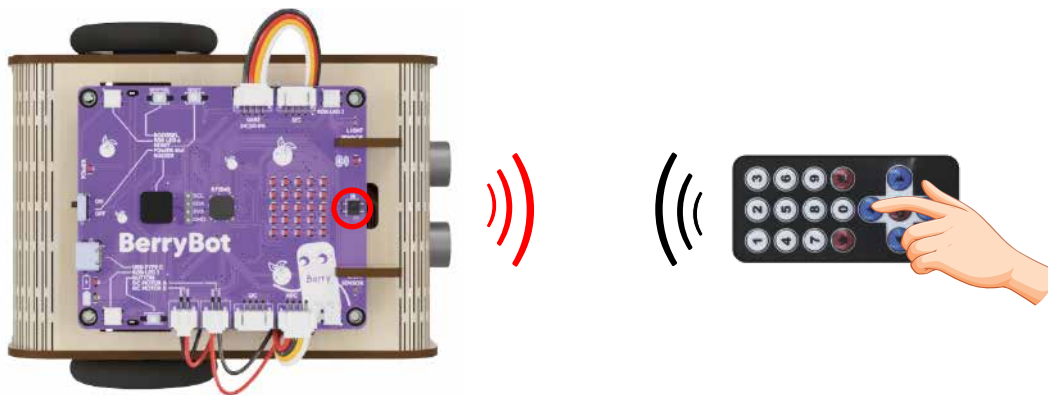
rbt.ist/lighttrackerarduino



Scan the QR code to
access the code.

IR Remote Control

In IR remote control mode, the BerryBot can be controlled using the buttons on the remote control included in the set and the IR receiver sensor on the mainboard.



The code blocks for the line tracker mode prepared using PicoBricks IDE are given in the image below.

Let's create the conditional structures that make the robot move forward when the up arrow button on the IR remote control is pressed, move backward when the down arrow button is pressed, move left when the left arrow button is pressed, and move right when the right arrow button is pressed.

After starting our code with the BerryBot block, let's create the infinite loop necessary for our code. At the beginning of the loop, add the stop block so that the motors stop if the direction buttons on the remote control are not pressed during each iteration of the loop.

Let's create the code blocks that stop the BerryBot if the direction buttons on the IR remote control are not pressed.





```
remote.py
1 #####Libraries#####
2 from machine import Pin, PWM, Timer, ADC
3 import time, utime
4 from berrybot import TB6612, NEC_ABC, NEC_16, IR_RX
5 #####Pin Defination#####
6 TX_PIN = 0
7 RX_PIN = 1
8 MOTOR_A1_PIN = 25
9 MOTOR_A2_PIN = 24
10 MOTOR_B1_PIN = 22
11 MOTOR_B2_PIN = 23
12 MOTOR_PWM_A_PIN = 15
13 MOTOR_PWM_B_PIN = 21
14 IR_PIN = 20
15 #####Variables#####
16 ir_data = 0
17 data_rcvd = False
18 #####Function Declaration#####
19 def ir_callback(data, addr, ctrl):
20     global ir_data
21     global ir_addr, data_rcvd
22     if data > 0:
23         ir_data = data
24         ir_addr = addr
25         print('Data {:02x} Addr {:04x}'.format(data, addr))
26         data_rcvd = True
27
28 ir = NEC_16(Pin(IR_PIN, Pin.IN), ir_callback)
29 motor = TB6612(MOTOR_A1_PIN, MOTOR_A2_PIN, MOTOR_B1_PIN, MOTOR_B2_PIN, MOTOR_PWM_A_PIN, MOTOR_PWM_B_PIN)
30
31 while True:
32     motor.stop()
33     if data_rcvd == True:
34         data_rcvd = False
35         if ir_data == IR_RX.number_up:
36             motor.forward(65535)
37             time.sleep_ms(500)
38         elif ir_data == IR_RX.number_down:
```

rbt.ist/remotemp



Scan the QR code to
access the code.



remote.ino

```
1 // libraries
2 #include <IRremote.h>
3
4 // Pin Definition
5 #define IR_PIN 28
6 #define PWM_A 15
7 #define PWM_B 21
8 #define MOTOR_B1 22
9 #define MOTOR_B2 23
10 #define MOTOR_A2 24
11 #define MOTOR_A1 25
12
13 //IR Button Numbers
14 #define number_1 69
15 #define number_2 70
16 #define number_3 71
17 #define number_4 68
18 #define number_5 64
19 #define number_6 67
20 #define number_7 72
21 #define number_8 21
22 #define number_9 9
23 #define number_8 25
24 #define button_up 24
25 #define button_down 82
26 #define button_right 90
27 #define button_left 8
28 #define button_ok 28
29
30 #define DECODE_NEC
31
32 void attachMotor()
33 {
34     pinMode(MOTOR_A1, OUTPUT);
35     pinMode(MOTOR_A2, OUTPUT);
36     pinMode(MOTOR_B1, OUTPUT);
37     pinMode(MOTOR_B2, OUTPUT);
38     pinMode(PWM_A, OUTPUT);
39     pinMode(PWM_B, OUTPUT);
40 }
41
```

rbt.list/remotearduino



Scan the QR code to
access the code.

BerryBot Pinout



RGB LED 5 (GPIO6)



GP10

Motor B



GPIO22
GPIO23



GPIO24
GPIO25

Motor A



I2C



ADC

Line Tracker
Left GP26
Right GP27



RGB LED 3 (GPIO6)

TX_RP2040
RX_RP2040

GPIO2
GPIO3

SDA
SCL

GPIO6
GPIO7

TX_RP
RX_RP

GP10
GPIO11

GPIO12
GPIO13

GPIO14
GPIO15

GND
XIN

XOUT
3V3

1V1
RESET

GPIO16
GPIO17

GPIO18
GPIO19

GPIO20
GPIO21

GPIO22
GPIO23

GPIO24
GPIO25

GPIO26
GPIO27

GPIO28
GPIO29

GP14

BUZZER

RP2040

(GPIO6) RGB LED 6



Ultrasonic Distance Sensor
Trig GP8
Echo GP9

UART

I2C

(GPIO6) RGB LED 2



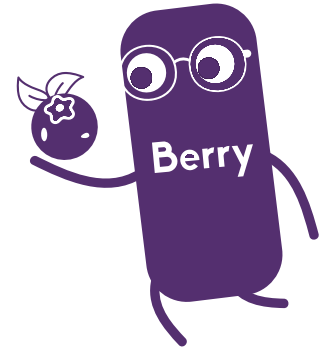
LDR
Left
GP29



IR GP20

LDR
Right
GP28

BerryBot



RobotistanINC

Selim GAYRETLI (Content) - Mehmet Suat MORKAN (Editor) - Atakan OZTURK (Hardware) -
Elanur Tokalak (Designer) - Sercan OKAY (Instrial Designer)

3 Germany Drive STE 4 #1430 Wilmington, DE 19804
hello@robotistan.com